Cuestionario

1. Conceptos del ABB y análisis del árbol construido

- **¿Qué propiedad debe cumplir todo Árbol Binario de Búsqueda?**
  Para todo nodo en el árbol, todos los valores almacenados en su subárbol izquierdo deben ser menores que la clave del nodo, y todos los valores almacenados en su subárbol derecho deben ser estrictamente mayores.
- **¿Cuál es la raíz del árbol construido?**
  La raíz es el nodo con el valor 50.
- **¿Qué nodos son hojas?**
  Los nodos hoja (aquellos que no tienen hijos) son 20, 40, 60 y 80.
- **¿Qué valores pertenecen al subárbol izquierdo de 50 y cuáles al derecho?**
  - **Subárbol izquierdo:** 30, 20, 40.
  - **Subárbol derecho:** 70, 60, 80.
- **¿Qué secuencia esperas obtener con el recorrido inorden?**
  Una secuencia ordenada de menor a mayor: 20, 30, 40, 50, 60, 70, 80.

2. Método de Búsqueda

- **¿Por qué no es necesario recorrer todos los nodos del árbol para buscar una clave?**
  Porque la propiedad del ABB permite descartar la mitad del árbol en cada paso: si el valor buscado es menor que el nodo actual se va a la izquierda, y si es mayor se va a la derecha.
- **Si se busca 40, ¿qué nodos se visitan y en qué orden?**
  $50 \rightarrow 30 \rightarrow 40$.
- **Si se busca 90, ¿qué condición permitirá concluir que no existe?**
  Al navegar ($50 \rightarrow 70 \rightarrow 80$), desde el nodo 80 se intentará avanzar hacia su hijo derecho. Al encontrar que la referencia actual es null, se concluye que la clave no existe.
- **¿Qué valor booleano debe regresar el caso base cuando el nodo actual es null?**
  Debe retornar false.
- **¿Qué ocurriría si el árbol no respetara la regla menor-izquierda y mayor-derecha?**
  La búsqueda binaria fallaría, obligando a realizar una búsqueda exhaustiva por todo el árbol sin garantía de encontrar la clave.

3. Método de Eliminación

- **¿Por qué la eliminación requiere más casos que la búsqueda?**
  Porque al eliminar un nodo intermedio se rompen las conexiones del árbol y es necesario reestructurar los punteros/referencias de los hijos para preservar la propiedad del ABB.
- **¿Qué debe ocurrir si la clave que se desea eliminar no existe?**
  El método alcanza un caso base null, no realiza ninguna modificación en la estructura y retorna null (o el subárbol sin cambios).

- **¿Por qué eliminar un nodo hoja es el caso más sencillo?**
  Porque no tiene hijos; simplemente se remueve devolviendo null a su nodo padre.
- **Si un nodo tiene solamente un hijo, ¿por qué puede devolverse directamente la referencia a ese hijo?**
  Porque ese único hijo hereda la posición del nodo eliminado reconectándose directamente con el padre del nodo eliminado sin violar el orden del árbol.
- **¿Por qué el menor valor del subárbol derecho es un candidato adecuado para sustituir a un nodo con dos hijos?**
  Porque el sucesor inorden (mínimo del subárbol derecho) es mayor que todos los nodos del subárbol izquierdo y menor que todos los demás nodos del subárbol derecho.
- **Después de copiar el valor sustituto, ¿por qué todavía es necesario eliminar ese valor de su ubicación original?**
  Para evitar tener claves duplicadas dentro del árbol.
- **¿Qué riesgo existiría si se eliminara un nodo con dos hijos sin reconectar correctamente sus subárboles?**
  El árbol perdería la referencia a uno o ambos subárboles, provocando pérdida de datos (fuga de memoria) y desconexión de la estructura.
- **¿Por qué eliminar la raíz puede modificar la variable raiz del árbol?**
  Porque la referencia principal raiz apunta directamente al nodo superior. Si este se elimina, la variable debe actualizarse para apuntar al nuevo nodo que toma su lugar.
- **¿Qué propiedad debe seguir cumpliendo el árbol después de cualquier eliminación?**
  La propiedad fundamental del ABB (menores a la izquierda, mayores a la derecha) y conservar el orden ascendente en el recorrido inorden.

4. Método Auxiliar (Valor Mínimo)

- **¿Hacia qué dirección debes desplazarte para encontrar el mínimo?**
  Siempre hacia la izquierda de forma recursiva o iterativa.
- **¿Qué condición indica que ya encontraste el nodo mínimo?**
  Cuando el nodo actual ya no tiene hijo izquierdo (nodo.izquierdo == null).
- **¿Cuál es el mínimo del subárbol cuya raíz es 70 en el árbol inicial?**
  El valor es 60.

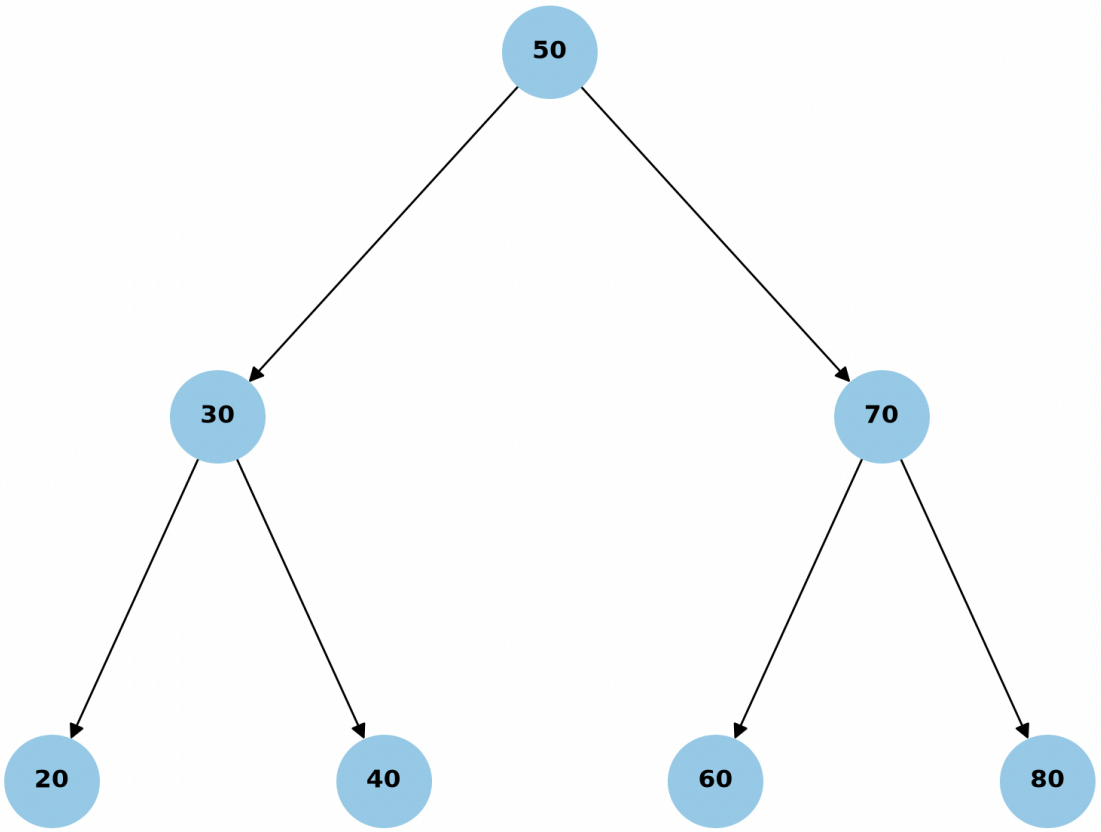
5. Reflexiones Finales

- **¿Cómo ayuda el recorrido inorden a comprobar que el ABB conserva su estructura?**
  Debido a que el recorrido inorden procesa los nodos en la secuencia *Izquierda - Raíz - Derecha*, si la estructura del ABB es correcta, el resultado siempre será una lista estrictamente ordenada de menor a mayor.
- **Explica el caso de eliminación que consideraste más difícil:**
  El caso de eliminación de un nodo con dos hijos, ya que requiere dos pasos clave: encontrar el sucesor inorden para reemplazar la clave, y luego eliminar recursivamente dicho sucesor de su posición original.
- **¿Qué papel cumple la recursividad en los métodos de búsqueda y eliminación?**

Permite descomponer el problema reduciendo el árbol a subárboles más pequeños en cada llamada hasta llegar a los casos base.

- **¿Qué aprendiste sobre el cambio de referencias entre nodos al eliminar elementos?**
  Que la eliminación en un árbol no destruye directamente las conexiones, sino que reasigna los punteros de retorno de cada llamada recursiva para mantener la jerarquía intacta.
- **¿Cómo explicarías la diferencia entre buscar y eliminar en un ABB a un compañero?**
  - **Buscar:** Es un proceso de solo lectura; navegas comparando valores hasta encontrar el nodo o llegar a null.
  - **Eliminar:** Requiere buscar el nodo primero y, una vez localizado, reestructurar las conexiones y punteros entre nodos para que el árbol no quede roto.

EVIDENCIAS DE FUNCION



```
--- ESTRUCTURA DE NODOS ---
Raíz: 50
Hijos de 50 -> Izq: 30 | Der: 70
Hijos de 30 -> Izq: 20 | Der: 40
Hijos de 70 -> Izq: 60 | Der: 80
-------------------------
```